Leopard SRS & Project Plan

Version 1
Group 18
COMPSCI 4ZP6
Dr. Mehdi Moradi
Oct 10, 2025

3. Personnel

Person	Email	Role
Krish Krish	krishk17@mcmaster.ca	Manager, Documentation, Developer, QA Tester
Rachelle De Man	demanr@mcmaster.ca	Developer, QA Tester, UI/UX
Arian Ahmadinejad	ahmadina@mcmaster.ca	Developer, QA Tester
Felix Fong	fongf2@mcmaster.ca	Developer, QA Tester, UI/UX
Timothy Leung	leungt30@mcmaster.ca	Developer, QA Tester

4. Glossary

Term	Definitions
VSCode	Visual Studio Code
IDE	Integrated Development Environment
Convex	Real time database and API service
Monaco	Open source code editor package
System	The completed form of the project being delivered
K8s	Kubernetes

5. Table of Content, Contributions

5.1 Table of Contents

3. Personnel..... 2
4. Glossary..... 2
5. Table of Content, Contributions..... 3
5.1 Table of Contents..... 3
5.2 Contributions Table..... 4
6. Project Purpose..... 5
6.1 Background..... 5
6.2 Problem Statement..... 5
6.3 Objectives..... 5
6.4 Planned/Proposed Solution..... 5
7. The Client, and other Stakeholders..... 6
7.1 Primary Stakeholders..... 6
7.2 Secondary Stakeholders..... 6
8. Project Constraints..... 6
8.1 Budget Constraints..... 6
8.2 Schedule Constraints..... 6
8.3 Technological constraints..... 6
9. Functional Requirements..... 7
9.1 P0 - Critical (Must Have)..... 7
9.2 P1 - High (Should Have)..... 7
9.2 P2 - Medium (Nice to Have)..... 7
9.3 P3 - Low (Optional / Future)..... 7
10. Data and Metrics..... 7
11. Non-functional Requirements..... 8
11.1 Look and Feel Requirements..... 8
11.2 Usability and Humanity Requirements..... 8
11.3 Performance Requirements..... 8
11.4 Security and Compliance Requirements..... 8
11.5 User documentation Requirements..... 9
12. Risks and Issues Predicted..... 9
Project Development Plan..... 10
13. Team Meeting and Communication Plan..... 10
14. Team Member Roles..... 10
14.1 Role Definition..... 10
14.2 Personnel responsibilities..... 10

15. Workflow Plan..... 10
15.1 Software and Pull Request Management..... 10
15.2 Agile Management..... 11
15.3 Data Storage..... 11
15.4 Computation Required..... 11
15.5 Logistic Plan to Achieve Requirements.....11
16. Proof of Concept Demonstration Plan..... 12
17. Technology..... 12
18. Project Scheduling: Gantt Chart..... 13

5.2 Contributions Table

Person	**Section(s)**
Krish Krish	3, 4, 5.1, 5.2, 6.2 – 6.4, 9.1 – 9.3, 10, 11, 15, 17, 18
Rachelle De Man	5.1, 7.2, 8.1, 8.2, 8.3, 8.4,11.2, 11.7, 18
Arian Ahmadinejad	5.2, 6.1, 12, 14, 15.3, 15.5, 17, 18
Felix Fong	6, 9, 11.1, 11.3, 11.4, 11.5, 11.6, 17, 18
Timothy Leung	6.3, 7, 9.1 - 9.3, 13, 14.2, 15.1 - 15.4, 16, 17, 18

6. Project Purpose

6.1 Background

Since the introduction of ChatGPT, students have started to AI generate code in lieu of doing assignments themselves. use it to complete all their assignments. As models have improved over time, it's now easier than ever to just upload assignments into ChatGPT, copy paste the output and submit that as-is.

To combat this, instructors are considering (or already moving) towards code-on-paper assignments. These go against the computer science spirit of hacking on code to learn, instead punishing students working authentically by making them toil with paper assignments. This also limits student learning by robbing them of building an understanding of code errors both syntactically and semantically. We believe coding on paper is a step in the wrong direction for education.

6.2 Problem Statement

The core problem is the lack of a reliable and transparent way to verify that submitted code was produced through legitimate student effort in a digital environment.

6.3 Objectives

Our objective is to introduce a system for post-secondary institutions that allows students to prove authenticity of their work in a code environment. This system must be accessible to all students, flexible enough to work for a variety of courses, and allow instructors to provision course assignments easily.

6.4 Planned/Proposed Solution

We propose to solve this problem with a web-based platform (the "system") that:

- Provides students with an IDE to complete assignments. This will allow them to view their assignment, edit and run code, and submit their work.
- Stores granular version history of student work to verify authenticity
- Analyze trends and detect malicious patterns commonly related with cheating (full copy/pasted solutions, typing out a final solution with no debugging) to flag AI use
- Provides a "replay viewer" for instructors to review flagged AI use within student code

7. The Client, and other Stakeholders

7.1 Primary Stakeholders

- Computer Software related Educators / Professors / TA's grading assignments who want to ensure their students don't use AI to complete it. They are primary stakeholders as they directly use our system to check on their students' work authenticity.
- Students taking a Computer Software related course are primary stakeholders as they will use our IDE to write coding assignments to prove their work's authenticity.

7.2 Secondary Stakeholders

- Academic Integrity boards are secondary stakeholders as they can use our system as evidence to prove cheating using AI occurred.
- Researchers interested in detecting AI coding are secondary stakeholders who may be interested in data trends regarding AI coding.

8. Project Constraints

8.1 Budget Constraints

- The use of the system should be free for students.
- The costs for upkeep of the system should be kept to a minimum.
- Data storage and fetching costs should be minimized.
- The cost for hosting the code running environment should be minimized.

8.2 Schedule Constraints

- We will need to have a functional "proof of concept" demonstration completed by November 21st, 2025.
- A design document for the project must be completed by January 23rd, 2026.
- We will need to complete the project sufficient for a final demonstration by March 27th, 2026.
- All applicable code must be uploaded to GitHub with a working user guide and finalized design document by April 4th 2026.
- A poster detailing the project for demonstration purposes must be completed by April 5th, 2026.

8.3 Technological constraints

- Our system must be usable on all computers (Mac, Windows, Linux).
    - Our system does not need compatibility for mobile devices.
- Our system must function on computers with varying computational ability.

- Our system must support popular browsers, i.e. Chrome and Firefox.

9. Functional Requirements

9.1 P0 - Critical (Must Have)

- User Authentication and Authorization: Students, Teaching Assistants and Professors need to be able to log in and only have access to their own resources. Different user roles should have access to different subsets of system features.
- Students must be able to create/edit/save code In a web based IDE.
- The system must keep a versioned record of every document.
- Instructors/admins should be able to view flagged/suspicious code submissions.

9.2 P1 - High (Should Have)

- TA/Professors can open a recorded session and watch a replay.
- Classroom Management: Instructors can intuitively create and manage classrooms as well as assignments
- Students should be able to run code through the IDE.
- The system should be resilient to infrastructure crashes as much as reasonably possible and should be able to recover with no human intervention.

9.2 P2 - Medium (Nice to Have)

- TA/Professors can view a student session live in real time.
- Autograding assessment using test cases.
- Authentication through McMaster emails / certain domain emails only.

9.3 P3 - Low (Optional / Future)

- Machine learning model / other advanced algorithms to pick up on cheating habits.

10. Data and Metrics

Our system will collect the following data:

- User Login and Permissions.
- Assignments (Set up by instructor).
- Student Submissions (with granular version history).

11. Non-functional Requirements

11.1 Look and Feel Requirements

- The IDE should feel like modern development tools (e.g. VSCode).
- The UI should be responsive where applicable.
- The UI design must be consistent (using a design system).
- The design should be tailored towards educators.

11.2 Usability and Humanity Requirements

- We will use easily readable fonts (ex. dyslexic friendly) and proper colour contrast to ensure readability.
- Proper zooming to increase font in code editor for easier readability.
- We will follow the general W3C guidelines to ensure proper accessibility is upheld.
- The system should have minimal user overhead for account creation, login and permission gathering where applicable.

11.3 Performance Requirements

- Editing (under optimal network conditions)
    - Instant editor input latency.
    - sub 1s codebase save time on every input.
    - sub 1s update times when viewing someone else's codebase edits.
- Cheating detection analytics
    - Individual submission - sub 20s for a review.
    - Batch submission - no strict time constraints. Within an hour ideally but the idea is to click a button and forget about it.
- Code execution
    - sub 60s code execution times.
- Reviewing
    - Sub 5s for viewing codebase snapshots.
    - Sub 1s for code diffing.
    - Smooth experience to view a project creation replay.
- should be able to scale to handle hundreds of concurrent users.

11.4 Security and Compliance Requirements

- Security, privacy, confidentiality
    - Ensure data is encrypted between requests.
    - Authentication on any API/data access.
- Processing integrity

    - Comprehensive testing.
    - ensure near 0 false positive rates (human written code flagged as AI generated).
    - Research to ensure high true positive rates.
- Keep cheating detection requirements confidential.
- Prevent malicious code from causing severe damage by executing them in isolated containers.

11.5 User documentation Requirements

We will provide annotated documentation from both a student and professor perspective.

- From a student standpoint it will detail how to run code.
- From a reviewer or professor standpoint it will explain how to view flagged students' work, navigate their code history, and use code history inspection tools.
- We will also teach student and professor users to navigate to their different classrooms they are enrolled in.

12. Risks and Issues Predicted

- Current AI-flagging software has false positives causing innocent students to lose grades and integrity. We need to be clear that the detection is for flagging and requires human review. It’s not a guaranteed marker for cheating.
- High latency or cost with code execution sandboxes, since if we roll our own infrastructure we can’t afford to keep VM’s running all the time and startup times may be high.

Project Development Plan

13. Team Meeting and Communication Plan

- We will be using discord primarily for all communications. We've set up a discord server where all the information and communication for this project should take place.
- We will be using Google calendar to schedule our meetings weekly since it shows our availability.
- To store any documents we'll be using Google Docs or tldraw.com depending on the nature of the document. These documents will also be linked on our discord for easier access.

14. Team Member Roles

14.1 Role Definition

The roles we will assign are as follows

- Project manager: This role defines the member who takes the lead role in breaking tasks into actionable items and ensuring timely completion of tasks.
- UI/UX Designer: This role defines the members who are taking a key role in deciding on the UI, UX, and aesthetics of the system.
- Developer: This role defines which members will be doing significant code additions and architecting the application.
- QA Tester: This role defines which members will be doing significant QA testing
- Documentation: This role defines which members will take the responsibility of creating relevant documentation for system users and developers.

14.2 Personnel responsibilities

- Krish will take on the responsibilities of Program Manager.
- Felix, Krish and Rachelle will take on the role of UI/UX Designer.
- Krish, Rachelle, Arian, Felix, and Timothy will all have the role of Developer.
- Krish, Rachelle, Arian, Felix, and Timothy will all have the role of QA tester.

15. Workflow Plan

15.1 Software and Pull Request Management

We will be using GitHub as our central workflow hub. This involves:

- Git for version control.
- GitHub Issues / Projects for issue tracking.
- GitHub pull requests for changes.

We also plan to use “Stacked Diffs”, a philosophy for making PRs in small incremental changes to make them easier to review. We will use https://graphite.dev/, which builds on top of native GitHub pull requests.

15.2 Agile Management

We will be using Github Projects to manage issues, tasks, and organize sprints to follow the agile method.

15.3 Data Storage

We will be storing our data in a relational database managed by Convex. Convex is a serverless backend platform, it allows us to get realtime updates and storage which is needed for our application. This is a convenience choice to increase developer speed.

15.4 Computation Required

Our platform allows users to run code as part of their development process. We plan to create sandbox code environments for users which are isolated containers to prevent malicious code execution. We can achieve this through building our own system or using sandbox providers. We will test the viability of each option and choose accordingly. Our options include:

- GCP Cloud Run Sessions
- E2B Sandboxes
- Modal Sandboxes
- Vercel Sandbox

There is no ML computation required for our project.

15.5 Logistic Plan to Achieve Requirements

- Strict Code Review: Every PR will be under scrutiny to ensure high quality code is being deployed.
- Strict Lint Rules
- Performance monitoring through:
    - Convex Dashboard for Backend; error rates, uptime, latency.
    - Posthog for Frontend Analytics: Frontend application errors, Google Lighthouse scores, etc.
- User monitoring through posthog:

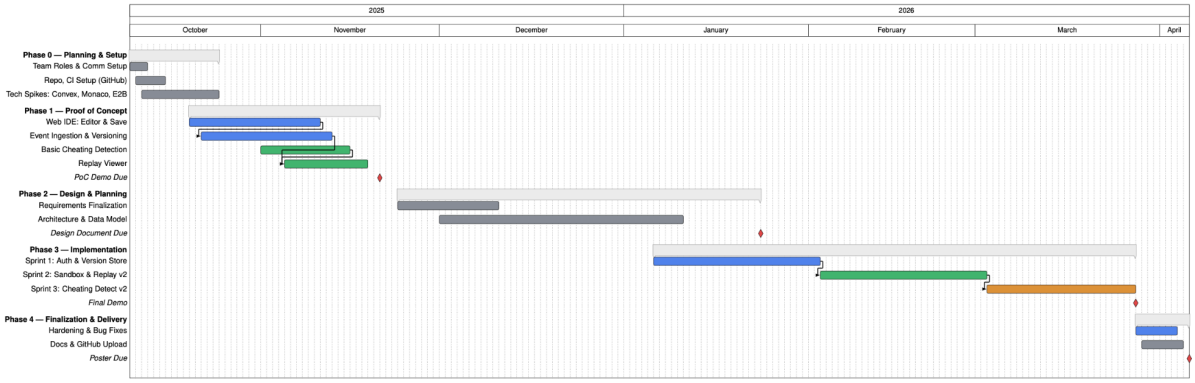
    - We will log events and monitor funnels to measure usage and user success on the platform. User success is defined as being able to complete and submit assignments.

16. Proof of Concept Demonstration Plan

By November 21st, we plan to deliver:

- A basic web based IDE with code editing functionality.
- Event ingestion based on code changes.
- Some dummy analysis on the ingested events to flag submissions.

17. Technology

- Frontend: React w/ Next.js
    - The team has experience with this technology.
    - It has strong Typescript support, which integrates well with the proposed backend solution.
    - Microsoft's Monaco Editor as base for the IDE.
- Backend: Node.js handlers served through Convex
    - Convex is a realtime database/API layer for Node.js backends that syncs data changes through websockets. We use this to reduce work required for realtime functionalities.
    - Authentication will be handled by better auth library, hosted through our Next.js application.
- CI/CD: Github Actions for CI
    - They are free and integrate well with our stack.
- Testing: Unit tests with Jest for React and Backend
    - Since we plan to write all of our code in a Typescript environment, Jest is a perfect choice for unit tests across the stack.
- Infrastructure:
    - Next.js app will be hosted on Vercel.
    - Convex can be self hosted or hosted through the official service. We will use the official service for smooth deployment and best practices.
    - We will be either using a third party code sandbox platform through E2B/Vercel/Modal or create our own infrastructure using K8s.
    - We will explore tools such as Pulumi, Terraform or Alchemy for infrastructure-as-code to manage our infrastructure requirements.

18. Project Scheduling: Gantt Chart